

TestMu AI Talent Assessment

SDET-2 | Quality Engineering Challenge

Overview

This isn't a theory test. We want to see how you actually design test systems. What patterns you reach for, how you handle scale and flakiness, and whether your framework would survive a real team using it.

There's no single right answer. We care more about your architectural judgment than the size of your test suite.

You can use AI tools (Claude, ChatGPT, Cursor, etc.) freely, but log them. We're not checking if you used AI; we're checking that you owned the design decisions.

Scenario

Imagine you've just joined the QA team at TestMu AI. We build a web-based test management platform and the team is shipping daily. Your first sprint ticket looks like this:

"Our regression suite is a mess. UI and API tests live in separate repos, there's no shared framework, half the tests are flaky, and nobody trusts the report. Build us a framework we can actually rely on, and wire it into something that surfaces signal: either a CI pipeline or an analytics view. Show us how you'd run quality, not just write tests."

Task 1: Setup and Scaffold

- **Repo:** Create a public GitHub repo named `testmu-sdet2-[yourname]`. This is where everything lives.
 - **Stack:** Playwright, Selenium, Cypress, or whatever you know well. Java, Python, or TypeScript. Your call.
 - **Structure:** Set up a clean folder structure with proper separation of concerns: pages, tests, utils, config, test data. Lay this out before you write a single test. Clean structure tells us how you think.
 - **First Commit:** Your first commit should show evidence of planning. A README skeleton, an architecture note, a folder scaffold, not just a `git init`. Tell us what you're building before you build it.
-

Task 2: Framework Architecture and Test Coverage

This is the core of the assignment. Build a test framework that another engineer on the team could pick up tomorrow and extend without asking you questions.

Coverage to build

- **UI Tests:** Login flow, dashboard interactions, form validations, cross-browser smoke. At least one test should run on more than one browser.
- **API Tests:** CRUD operations, authentication flows, error handling (4xx/5xx), schema validation, response-time assertions.
- **Integration Tests:** At least one end-to-end flow that combines UI and API layers (for example, create a record via API, assert it appears in the UI).

What we expect to see in the framework

- **Design pattern:** Page Object Model, or an equivalent abstraction you can defend. A base page, encapsulation, no raw selectors leaking into tests.
- **Data-driven approach:** Test data externalised (JSON, CSV, YAML, fixtures). Parameterised tests over multiple data sets.
- **Reusable utilities:** Custom waits, retry logic, custom assertions, config-driven environment switching.
- **Reporting:** Integrated test reports (Allure, ExtentReports, Playwright HTML, or equivalent) with screenshots on failure and logs attached.
- **Selectors:** Consistent, robust strategy such as `data-testid` or semantic locators, not

brittle XPath.

What to submit

- The framework code with the layout above.
 - `test-strategy.md` with your test approach, what you chose to cover and why, what you'd cover next, and the top three risks you'd flag to the team.
 - A sample test report (screenshot, HTML output, or hosted link) showing a passing run and a failing run with the failure artifact attached.
-

Task 3: DevOps and Quality Integration

Tests that nobody runs aren't tests. Pick one of the two options below and build it. It must be real and wired to your framework, not a config you copy-pasted from a tutorial.

Pick ONE

- **Option A, CI/CD Pipeline:** Configure a GitHub Actions or Jenkins pipeline that runs your test suite on push/PR, publishes the report as an artifact, and notifies on failure (Slack webhook, email, GitHub status check, your choice). Bonus if you parallelise or shard the run.
- **Option B, Test Analytics Dashboard:** Build a reporting layer that tracks trends across runs: pass/fail rate over time, flaky-test detection, slowest tests, failure clustering by module. Can be a static site, a notebook, a small web app, or a wired-up Grafana/Metabase view. The data must come from real runs of your framework, not synthetic data.

What to submit

- Working configuration or code for the option you picked.
- Documentation of how to set it up and run it locally or in CI.
- Sample output, screenshots, or a hosted link demonstrating it working end-to-end across more than one run.
- A short comment in your code or docs explaining why you picked this option over the other.

Submission

Share your GitHub repo link once done. Make sure it's public and contains all of the below:

- The complete framework with UI tests, API tests, integration tests, POM/design pattern, utilities, test data.
 - Test reports with screenshots on failure.
 - Task 3 deliverables: pipeline config or dashboard code, plus sample output.
 - `README.md` with architecture overview, setup, how to run, design decisions you made, and what you'd build next with more time.
 - `test-strategy.md` with coverage rationale, risk analysis, improvement plan.
 - `ai-usage-log.md` with every AI tool used, the task it helped with, and what it produced.
- Optional but encouraged. We trust senior engineers to own their tooling choices.
-

TestMu AI, SDET-2 Assessment